INVOLUTE GEAR: INFLUENCE OF THE TOOTH HEIGHT

MATLAB Code:

```
%Input values=====================================================
m=2;
z1=14;
z2=20;
x1= 0;
x2= 0;
alpha= 20;
alpha_r=alpha*pi/180;
phi=tan(alpha_r)-alpha_r;  %Involute of pressure angle

%Working pressure angle
%For angles upto 72 degrees, Cheng method
%Reference from Dudley
a= 2*((x1+x2)/(z1+z2))*tan(alpha_r) + tan(alpha_r)-alpha_r;
%display(['Involute working pressure angle= ',num2str(a)]);
alpha_w= 3^(1/3)* a^(1/3)-(2/5)*a+ (9/175)*3^(2/3)*a^(5/3)-(2/175)*3^(1/3)*a^(7/3)-
(144/67375)*a^(9/3)+(3258/3128125)*3^(2/3)*a^(11/3)-
(49711/153278125)*3^(1/3)*a^(13/3)-
(1130112/9306171875)*a^(15/3)+(5169659643/95304506171875)*3^(2/3)*a^(17/3);
%display(['Working pressure angle in degrees= ',num2str(u*180/pi)]);


center_d= (m*(z1+z2)/2)*((cos(alpha_r)/cos(alpha_w)));
%Geometry
pcr=(m*z1/2); %Pitch circle radius
rb=((m*z1)/2)*cos(alpha_r); %Base circle radius
pcr_w=rb/cos(alpha_w); %Working pitch circle radius
c=0.167*m;  %Manufacturing clearance
da= m*z1+2*m+2*m*x1; % Addendum diameter
ra=da/2; %Addendum radius
h=2.25*m; %Tooth depth
ha=m*(1+x1);
hf=m*(1.25-x1);
df=da-2*h;  % Dedendum diameter
rf=df/2;  %dedendum radius
```

```
s0=m*((pi/2)+2*x1*tan(alpha_r)); %Tooth thickness
rt=0.300*m; %Tool tip radius

%Data points to plot
theta=linspace(0,2*pi,100);

%Plotting of base circle
xb=rb*cos(theta);
yb=rb*sin(theta);

%Plotting of base circle
xp=pcr*cos(theta);
yp=pcr*sin(theta);

%Plotting of addendum circle
xa=ra*cos(theta);
ya=ra*sin(theta);

%Plotting of dedendum circle
xd=rf*cos(theta);
yd=rf*sin(theta);

%Calculation for one tooth of involute gear
ry=linspace(rb,ra,100);
%theta1=sty/ry;
alpha_y=acos(rb./ry);
sty=ry.*((s0/pcr)+ tan(alpha_w)-alpha_w - (tan(alpha_y)-alpha_y));

%involute curve
%xinv=ry.*sin(theta1/2);
%yinv=ry.*cos(theta1/2);

%Involute curve
xinv=ry.*sin((pi/(2*z1))+tan(alpha_w)-alpha_w-tan(alpha_y)+alpha_y);
yinv=ry.*cos((pi/(2*z1))+tan(alpha_w)-alpha_w-tan(alpha_y)+alpha_y);

%Involute curve mirror
xinv_mirror=-xinv;
```

```
yinv_mirror=yinv;

%Top Land
ry_top = ra; % Setting the variable radius at addendum or Top Land

alpha_y = acos(rb/ry_top); % Pressure angle for Top Land radius
xinv_top = ry_top * (sin((pi/(2*z1)) + tan(alpha_w)-alpha_w -...
    tan(alpha_y)+alpha_y));

x_top = linspace(-xinv_top,xinv_top,100);
y_top = sqrt(ra^2 - x_top.^2);

%Trochoid equation
ry1=rb;
alpha_y1=acos(rb/ry1);
%slope k at the transition point
k=((cos(pi/(2*z1))+phi-tan(alpha_y1)+alpha_y1)/(sin(pi/(2*z1))+phi-
tan(alpha_y1)+alpha_y1));

%Angle theta for the arc
theta1=atan(-1/k);

%Radius rc of the fillet arc
rc=(rb-rf)/(1+sin(theta1));

%Center of the fillet arc
xc=xinv(1)+rc*cos(theta1);
yc=yinv(1)+rc*sin(theta1);

%Coordinates of intersection of trochoid and root circle
point1 = (rf^2+xc^2+yc^2-rc^2)/2;
point2 = ((2*point1*xc)^2)-(4*(xc^2+yc^2)*(point1^2-((rf^2)*(yc^2))));
x_intersection = ((2*point1*xc)-sqrt(point2))/(2*(xc^2+yc^2));
y_intersection = sqrt(rf^2-x_intersection^2);
theta3 = asin((xc-x_intersection)/rc);

%Arc fillet on the tooth profile
theta_fillet = linspace(3*pi/2-theta3, pi+theta1, 100);
```

```
x_trochoid = xc + rc * cos(theta_fillet);
y_trochoid = yc + rc * sin(theta_fillet);

%Start and end points of trochoid
x_trochoid_start= x_trochoid(1);
y_trochoid_start= y_trochoid(1);
x_trochoid_end= x_trochoid(end);
y_trochoid_end= y_trochoid(end);

% %Bottom land
theta_intersect= atan(y_intersection/x_intersection);       %Finding the intersection point
theta_single_arc=((2*pi)-(2*z1*(pi/2-theta_intersect)))/z1;  %Optimization of the bottom
angle
theta_bot=theta_intersect-theta_single_arc;
theta_bottom_arc=linspace( theta_intersect, theta_bot,100);
x_bot=rf.*cos(theta_bottom_arc);
y_bot=rf.*sin(theta_bottom_arc);

%Plotting the curves
figure;
hold on
for i = 0:z1-1
   theta_i = (2*pi/z1) * i;  % Angle for each tooth
   rot_matrix = [cos(theta_i) -sin(theta_i); sin(theta_i) cos(theta_i)]; % Rotation Matrix

   % Generating involute curves coordinates after multiplying curves by rotation matrix
   inv_rotate = rot_matrix * [xinv; yinv];
   xinv_rot = inv_rotate(1, :);
   yinv_rot = inv_rotate(2, :);
   plot(xinv_rot,yinv_rot,'k-')
   plot(-xinv_rot,yinv_rot,'k-')

   % Generating top land coordinates after multiplying curves by rotation matrix
   top_rotate = rot_matrix * [x_top; y_top];
   x_top_rot = top_rotate(1, :);
   y_top_rot = top_rotate(2, :);
   plot(x_top_rot,y_top_rot,'k-')
   plot(-x_top_rot,y_top_rot,'k-')
```

```
    % Generating non-involute curve coordinates after multiplying curves by rotation matrix
    trochoid_rotate = rot_matrix * [x_trochoid; y_trochoid];
    x_trochoid_rot = trochoid_rotate(1, :);
    y_trochoid_rot = trochoid_rotate(2, :);
    plot(x_trochoid_rot,y_trochoid_rot,'k-')
    plot(-x_trochoid_rot,y_trochoid_rot,'k-')


    %Generating bottom land coordinates
    bottom_rotate = rot_matrix * [x_bot; y_bot];
    x_bottom_rot = bottom_rotate(1, :);
    y_bottom_rot = bottom_rotate(2, :);
    plot(x_bottom_rot,y_bottom_rot,'k-')


end

%Plot
hold on
plot(xb,yb,'r.')
plot(xa,ya,'b--')
%plot(xd,yd,'k-')
plot(xp,yp,'g--')
%plot(x_bot,y_bot,'k-')


plot(xinv,yinv,'k')
plot(xinv_mirror,yinv_mirror,'k')
%plot(xc,yc,'k*')
plot(x_trochoid,y_trochoid,'k')
plot(x_top,y_top,'k')
%plot(x_combined,y_combined,'c')
hold off
axis equal
```